

Campus equipment lending manager

CampusKit

A terminal application for inventory, borrowers, circulation and overdue reporting.

Course and student

Course: Programming in Java

Name: Ayush Yadav

Registration number: 25BAI10946

Report date: 18 September 2026

Project summary

Ten Java application source files implement four functional modules with local file persistence. The application requires only JDK 17 or newer and runs without a graphical setup or external Java dependencies.

Authorship and review note

The implementation and this report were prepared with AI assistance. The student should review, understand and personalize the work and ensure that its use follows the course policy. This report documents implemented behavior and observed checks; it does not assert unaided authorship or invented personal learning experiences.

Contents

- 2 Requirements and scope
- 3 Architecture and use cases
- 4 Workflow and sequence
- 5 Class and storage design
- 6 Implementation and decisions
- 7 Execution results and testing
- 8 Challenges, learning topics and references

Requirements and scope

1. Introduction and problem statement

Campus clubs and laboratory desks share calculators, cameras and similar equipment. A paper register can make availability and overdue returns difficult to track. CampusKit records each loan against a member and equipment type and derives current availability from active loans.

2. Objectives and target users

Provide a reliable, understandable terminal workflow for lending-desk operators. Preserve records across restarts, reject invalid borrowing requests and identify late returns. Borrowers are registered members; they do not log in to the application.

3. Functional requirements

Inventory: register named equipment types with positive quantities and list availability.

Membership: register and list members using generated IDs.

Circulation: issue loans for 1-30 days, allow at most three active loans per member, reject overdue borrowers, record returns and retain history.

Reporting: show members, equipment types, total/available units, active loans and overdue loans with days late.

4. Non-functional requirements

Portability: Java 17 target and standard library only.

Reliability: atomic snapshot replacement; failed writes leave live state unchanged.

Integrity: validate IDs, dates, references and stock on load.

Usability: numbered menus, validation messages and safe end-of-input handling.

Maintainability: distinct UI, service, domain and repository responsibilities.

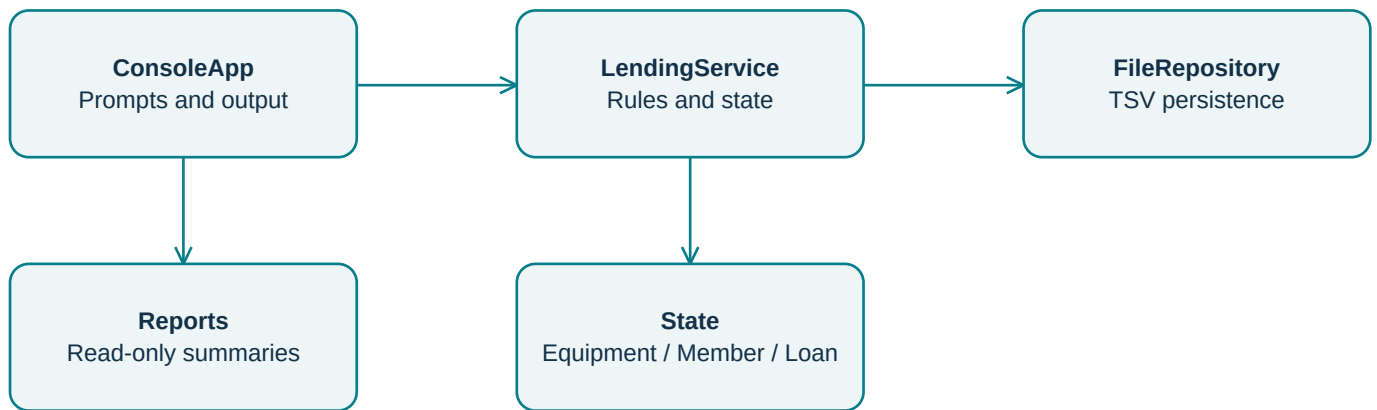
Concurrency: one process per data directory using a file lock.

5. Boundaries

Intended for small local registers. No login, payments, fines, reservations, network server, deletion or serial-number tracking is implemented. Quantities represent interchangeable units. Performance at large scale has not been benchmarked.

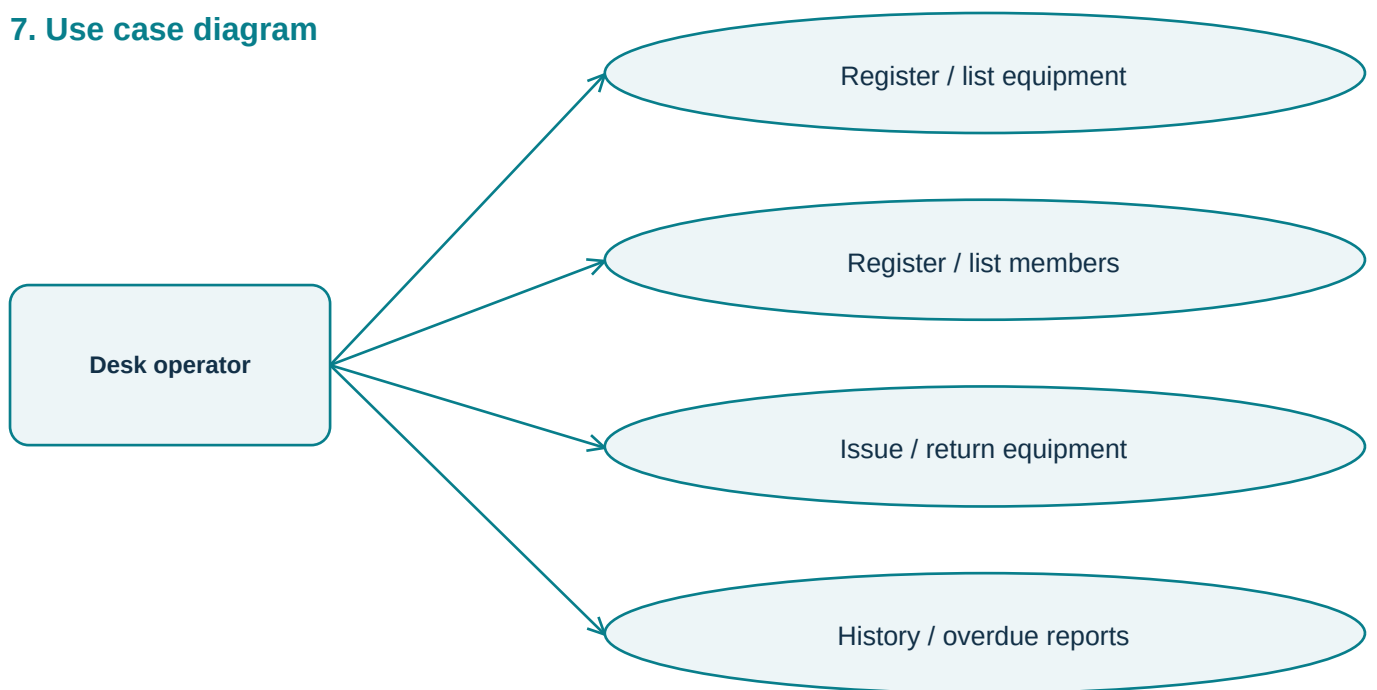
Architecture and use cases

6. Layered architecture



Main creates the data directory and obtains an exclusive lock before loading data. LendingService depends on the Repository interface; FileRepository implements the persistence contract. Reports consumes a copied state and a date.

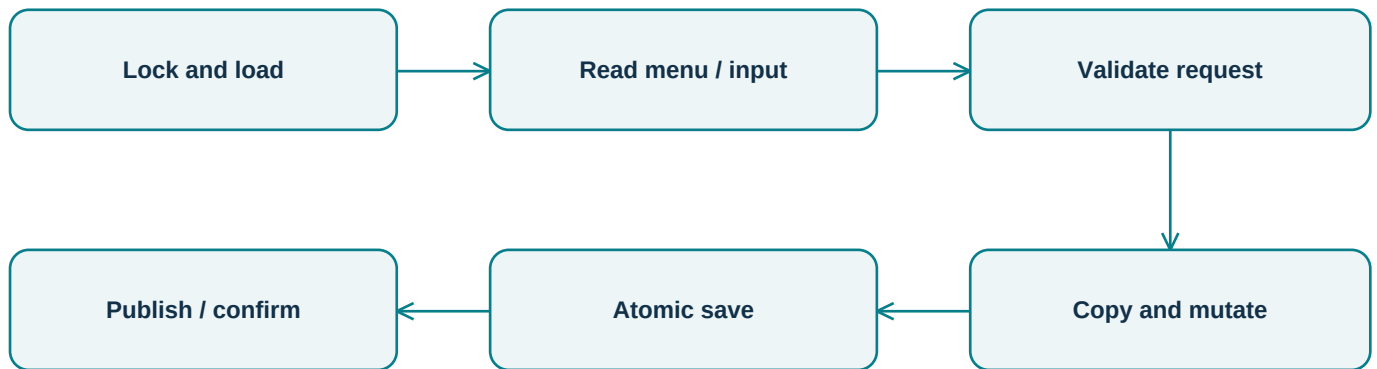
7. Use case diagram



The operator performs all use cases on behalf of members. Stock and borrower validation are included in the issue-equipment workflow.

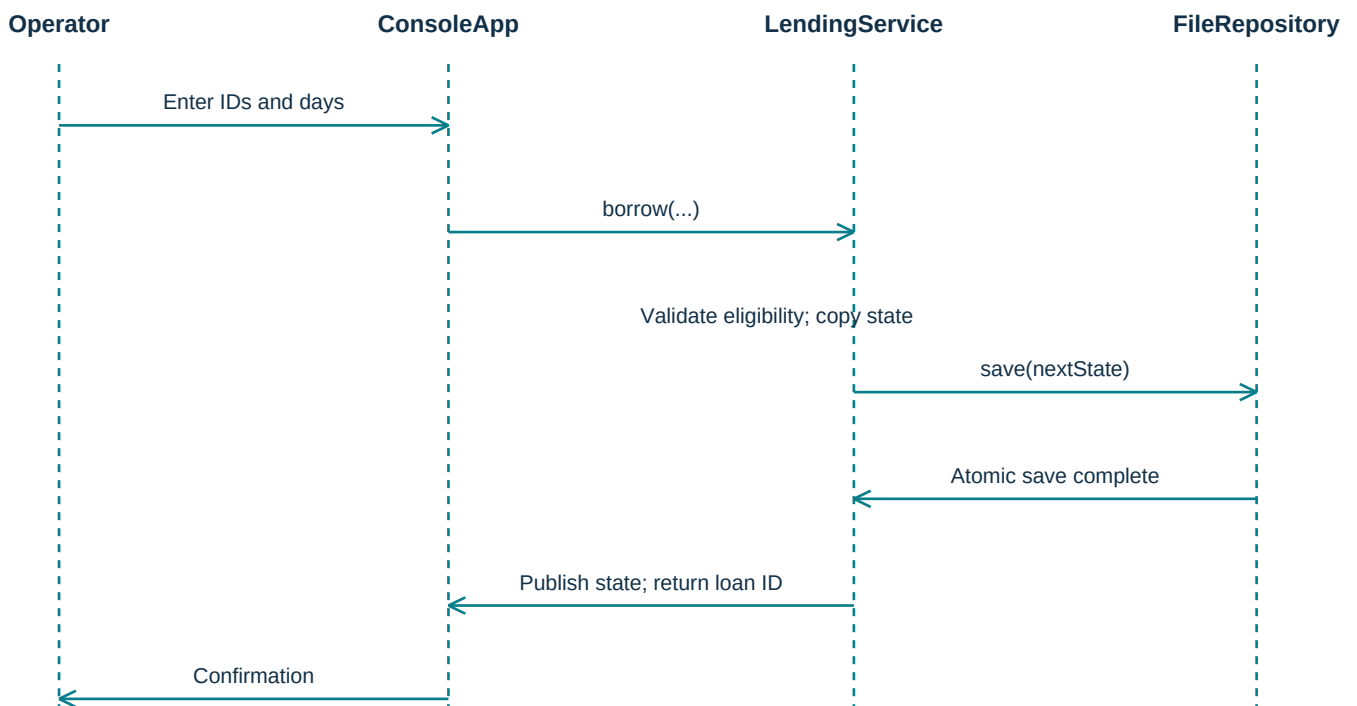
Workflow and sequence

8. Request workflow



Validation errors return to the menu. A save failure reports an error and retains the old state. Read-only requests display output without saving. Exit or end-of-input releases the lock. Startup load failures stop execution without overwriting the data.

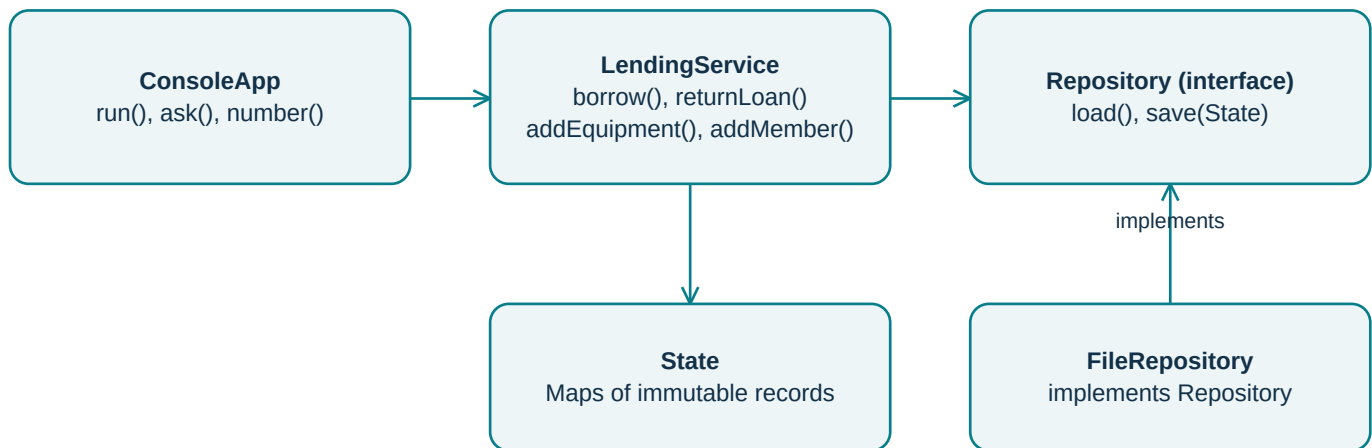
9. Successful borrow sequence



Failure path: validation or IOException propagates to the console; successful state publication occurs only after repository.save returns.

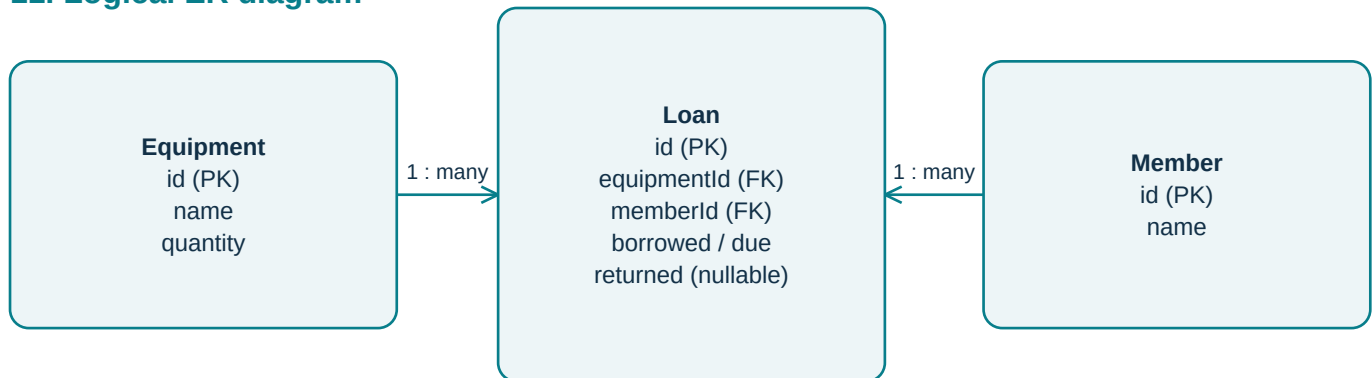
Class and storage design

10. Class / component diagram



Equipment, Member and Loan are immutable Java records. State copies its maps while sharing these immutable values. Reports formats read-only summaries. Main wires dependencies and owns the lock lifecycle. Repository supports alternate persistence implementations and simulated failures in tests.

11. Logical ER diagram



12. Storage schema

UTF-8 file: campuskit.tsv. Header: CAMPUSKIT followed by a tab and version 1. E rows store id, encoded name and quantity; M rows store id and encoded name; L rows store id, equipmentId, memberId, borrowed, due and returned. All fields use tabs. Names use Base64; dates use ISO YYYY-MM-DD. An empty returned field marks an active loan.

Load checks reject duplicate IDs, invalid dates, missing referenced records and active loans exceeding stock. No SQL engine is required. Data is not encrypted; local filesystem permissions govern access.

Implementation and decisions

13. Java concepts demonstrated

Encapsulation: LendingService keeps state private and exposes copied snapshots.

Abstraction and polymorphism: Repository defines load/save; the file implementation and a test double share the contract.

Records and collections: immutable entities live in ordered LinkedHashMap collections.

Exception handling: validation uses IllegalArgumentException; persistence uses IOException; the console recovers from ordinary input errors.

File handling: NIO paths, temporary files, atomic moves and process locks.

Date processing: LocalDate, ChronoUnit and injectable Clock support deterministic due-date logic.

14. Decisions and rationale

Availability is derived from active loans instead of maintaining a separate mutable stock counter. A complete copied state is saved before being published in memory, preventing disk failures from leaving the session ahead of the saved file. Names are encoded to avoid delimiter ambiguity. CLI-only operation satisfies terminal execution requirements and avoids UI framework setup.

15. Complexity and tradeoffs

Memory and snapshot storage grow as $O(E+M+L)$. A save and state copy are linear in the number of records. Borrow eligibility scans loans: $O(L)$. Listing inventory is $O(E*L)$ because each equipment type calculates availability from loans. These choices suit small registers but motivate indexed storage for larger deployments.

16. Setup and execution

Install JDK 17 or newer and make java and javac available on PATH. No third-party Java dependencies or configuration secrets are needed. From the repository root:

```
javac --release 17 -d build "@sources.txt"
```

```
java -cp build campuskit.Main
```

An optional final argument selects the data directory. See README.md for Windows and POSIX scripts, a first-session walkthrough and recovery guidance.

Execution results and testing

17. Observed terminal results

The captured demo registered two Scientific Calculator units and one Demo Student, issued one seven-day loan, displayed availability, recorded its return and retained loan history. The following lines are from the actual terminal run:

```
1 | Scientific Calculator | 1 / 2
```

```
Total units: 2 | Available: 1 | Active loans: 1 | Overdue: 0
```

```
Choice: Loan ID: Return recorded.
```

```
Total units: 2 | Available: 2 | Active loans: 0 | Overdue: 0
```

18. Test method and outcome

All 25 checks passed. Tests were executed on Windows with JDK 26, compiling against the Java 17 API target. A fixed UTC clock makes overdue tests reproducible; temporary directories isolate file tests. Assertions fail the test process on an unexpected result.

Checks cover empty startup, invalid names and quantities, unknown IDs, duration limits, exhausted stock, due-date boundaries, overdue restrictions, returns, the three-loan cap, reload, Unicode persistence, snapshot isolation, failed-save rollback, console recovery, dangling references and corrupt-file preservation.

Evidence: docs/test-results.txt and docs/demo-output.txt. Run:

```
javac --release 17 -d build "@sources.txt" tests/campuskit/ProjectTests.java
java -cp build campuskit.ProjectTests
```

19. Validation limits

The test suite is targeted rather than exhaustive. It does not simulate power loss or benchmark large datasets. Atomic replacement is filesystem-dependent. A GitHub Actions workflow is included to compile and test on Ubuntu with Java 17; its actual remote result must be inspected after publication.

Reflection and references

20. Engineering challenges addressed

Stock consistency was addressed by calculating availability from loan records. Failed saves were addressed by copying state and publishing it only after persistence succeeds. Date-dependent testing was addressed with Clock injection. Corrupt input files trigger a startup error instead of silent recovery that could discard records. These describe engineering concerns handled by the implementation, not a claim about the student's personal development history.

21. Learning topics and key takeaways

Review how interface-based design enables failure injection, why immutable records simplify copying, how derived state avoids duplicated counters, and how exception boundaries separate user errors from storage failures. Before evaluation, the student should be able to explain the borrow/return flow, demonstrate tests and discuss the tradeoffs of flat-file persistence.

22. Future enhancements

Add serial-number tracking, reservations, controlled editing, CSV export and indexed database storage. A multi-user version would require authentication, authorization, transactional persistence and conflict handling. Add platform-specific tests and fault-injection checks before wider deployment.

23. References

1. VITyarthi, Build Your Own Project: General Project Instructions & Submission Guidelines (user-provided PDF).
2. Java SE 17 API documentation: <https://docs.oracle.com/en/java/javase/17/docs/api/>
3. Java language specification, Java SE 17: <https://docs.oracle.com/javase/specs/jls/se17/html/>
4. Project source, README.md, statement.md, docs/design.md and test evidence in this repository.

24. Submission checklist

Use a public GitHub repository containing the root README, statement, source files, tests and design documentation. Submit the repository root URL rather than a tree/blob link. Upload this PDF separately through the course portal. Review the course policy on AI assistance and complete any required personal review before submission.